Table 1: The IMPLY logic gate's truth table.

Case	p	q	$p \rightarrow q$
1	0	0	1
2	0	1	1
3	1	0	0
4	1	1	1

The coordination of the applied architecture and the design method with the crossbar array structure is one of the essential features to consider when designing memristive arithmetic circuits for PIM. Implementing arithmetic circuits based on parallel architecture is not recommended due to the complexity and incompatibility with the crossbar array [2, 16]. Semi-serial and semi-parallel architectures have moderate compatibility with the memristive crossbar array [2]. The proposed circuits of this article and the reviewed SOA in the following subsections are only based on the serial architecture due to its simplicity and compatibility with the crossbar array, and other architectures will not be reviewed.

2.2 IMPLY-based components of the multiplier

After a thorough investigation of standard multipliers, it is determined that three primary stages must be implemented to compute the final product. The first step is Partial Product Generation (PPG). AND and NAND gates can be applied to generate the partial products in unsigned and signed multipliers. In the second stage of implementing the multiplier, Partial Product Reduction (PPR), half adder, full adder, and compressor cells are applied to simplify the multiplication tree. Several algorithms have been introduced to reduce the multiplication tree, one of the most important of which is the Dadda reduction algorithm [27]. In this article, half adder and full adder cells proposed in [23, 28] are applied to reduce the multiplication tree. In the third stage, the final product is calculated by the Ripple Carry Adder (RCA), in which inputs are the output of the second stage. In the structure of add & shift and radix-2 Booth multipliers, multiplexers are needed in addition to full adder cells.

Table 2 summarizes the features of basic circuits required to implement unsigned and signed classic multipliers. The total number of memristors (input/output and work), computational steps, and the possibility of reusing the input memristors to store the output(s) are written for each memristive circuit.

Table 2: Characteristics of basic circuits required to implement classic unsigned and signed multipliers.

Circuit	No. of Steps	Total No. of Memristors	Input Memristor Reusability
NOT(a) [23]	2	2	–
AND(a, b) [23]	5	3–4	+
NAND(a, b) [23]	3	3	+
Half Adder(a, b) [28]	12	4	+
Full Adder(a, b, C_{in}) [23]	22	5	+
Copy(a)	4	3–4	+
XOR(a, b) [23]	9	4	+
First MUX2:1(a, b, *Select*)	9	6	–
Second MUX2:1(a, b, *Select*)	7	5	–

2.3 IMPLY-based serial multipliers

In [28], modified IMPLY-based serial 4-bit and 8-bit multipliers are introduced. The authors' main idea in [28] was to present a fast and energy-efficient memristive 4:2 compressor. The outputs of this compressor are computed in only 44 computational steps by applying seven memristors. The 4:2 compressor proposed in [29] needs 52 computational steps and seven memristors, the same as the circuit proposed in [28], to compute the

outputs. The PPR and RCA stages of the proposed multipliers in [28] are merged by applying the serial architecture. The n-bit implementation of the multiplier requires $n^2 + 2$ memristors to compute the final product in $27n^2 - 32n$ computational steps [28].
To the best of our knowledge, the only fully serial multiplier based on IMPLY logic has been introduced in [28]. Considering this limitation, the authors have implemented signed and unsigned conventional multipliers by applying the basic cells introduced in subsection 2.2 based on IMPLY logic and serial architecture. So, the proposed unsigned and signed multipliers can be compared with the classic designs. It is tried to use the minimum number of work memristors in addition to the input/output memristors to implement the classic multipliers. Moreover, the authors tried to minimize the steps of the multipliers' implementation algorithms and consider possible overlaps between the computational steps of different applied basic cells, such as the full adder proposed in [23]. The specifications of the presented unsigned and signed multipliers are reported in Tables 3 and 4, respectively. The number of computational steps and required memristors (input/output and work memristors) for implementing the n-bit multipliers are also written in Tables 3 and 4. It should be mentioned that circuits designed in serial architecture do not need CMOS switches for proper functionality. Hence, the number of CMOS switches required by the multipliers and their components is zero.

Table 3: Characteristics of classic unsigned multipliers.

Classic Multiplier	No. of Steps	Total No. of Memristors
[28]	$27n^2 - 32n$	$n^2 + 2$
Dadda	$27n^2 - 32n$	$n^2 + 2$
Add & Shift	$31n^2 + n + 4$	$3n + 5$

Table 4: Characteristics of classic signed multipliers.

Classic Multiplier	No. of Steps	Total No. of Memristors
Radix-2 Booth	$49n^2 + 15n - 4$	$4n + 8$
Baugh-Wooley (Dadda)	$27n^2 - 24n + 24$	$n^2 + 2$
Add & Shift	$31n^2 + n + 4$	$3n + 5$

It should be noted that some of the combinational logic circuits applied in classic multipliers are implemented without any optimization and overlapping computational steps. The 2:1 multiplexers and carry generation circuit applied in add & shift and radix-2 Booth multipliers are among these designed combinational circuits.

3 Proposed IMPLY-based serial unsigned and signed multipliers

3.1 Unsigned serial array multiplier

The array multiplier is known for its well-organized structure. It consists of partial product units (PPUs) made up of full adders and half adders arranged in an array (rows and columns) next to each other. Figure 2 depicts the structure of a 4-bit unsigned array multiplier based on CSA.
The unsigned array multiplier's outputs are obtained by applying four arithmetic blocks. The output's least significant bit (LSB) is calculated using the AND function of the inputs' LSB, x_0 and y_0. The other output bits of this multiplier are computed by applying three different PPUs (PPU_1, PPU_2, and PPU_3) in addition to a half adder and some full adders. Each of these PPUs' structure are specified with different color in Figure 2. Principally, PPUs are placed in n rows and $n - 1$ diagonal lines in the n-bit unsigned array multiplier. The PPU_1 is an arithmetic unit that uses two AND gates to generate partial products and a half adder to add these partial products. The PPU_2 is another arithmetic unit consisting of one full adder cell applied to sum up a partial product generated by an AND gate with the outputs of the previous stage's PPUs. Similarly, two AND gates and a full adder cell are the building blocks of the PPU_3. 4, 3, and 4 memristors (input and work memristors) are needed to implement the PPU_1, PPU_2, and PPU_3 by applying the serial IMPLY logic, respectively, based on the basic gates and arithmetic cells listed in Table 2.

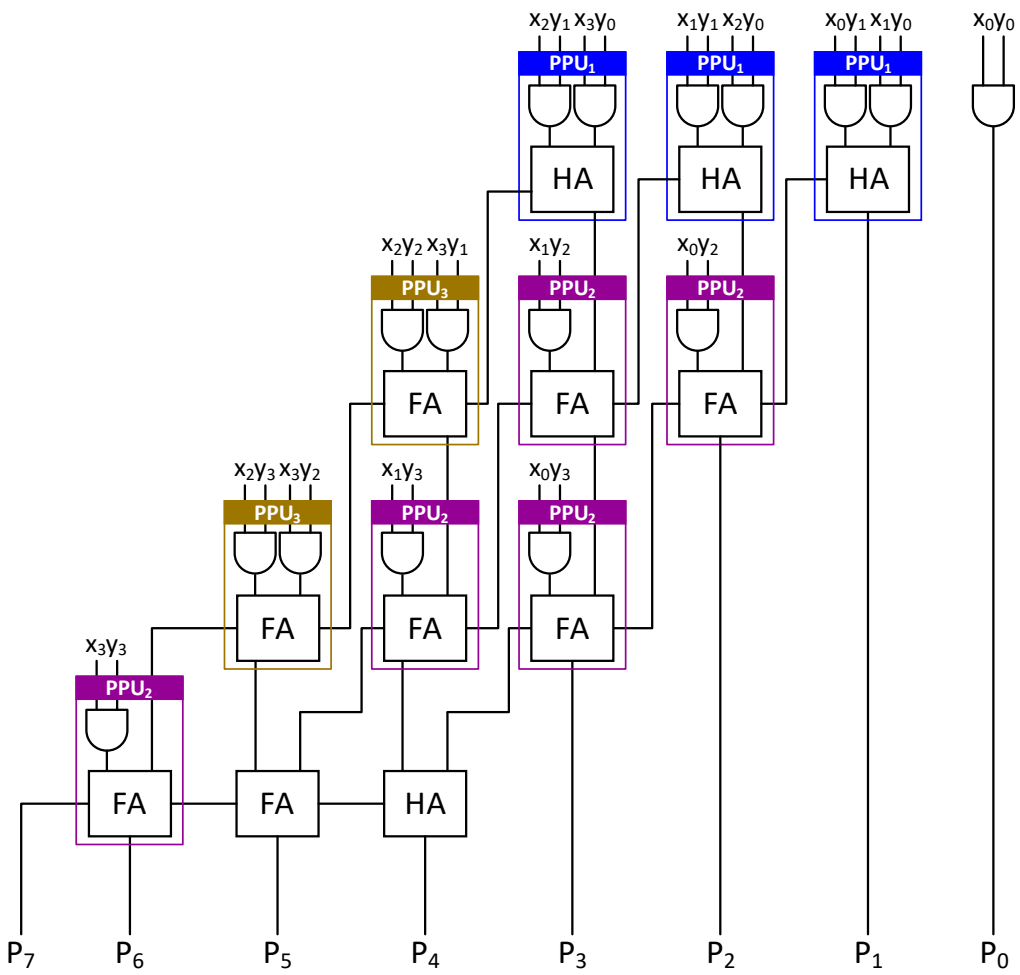

Figure 2: 4-bit unsigned CSA-based array multiplier.

The number of required building blocks for implementing the classic n-bit unsigned array multiplier as well as their computational steps is reported in Table 5. Implementing each building block using IMPLY logic in the serial architecture requires 5 (AND gate) to 32 (PPU_3) computational steps, according to the steps reported in Table 5. Therefore, $27n^2 - 32n$ computational steps and $5n - 4$ memristors ($2n$ input memristors and $3n - 4$ work memristors) are required to implement the classic n-bit unsigned array multiplier. The outputs of the classic serial unsigned array multiplier implemented based on IMPLY logic are written in the input memristors, and no extra memristor is needed to store the outputs.

Table 5: Characteristics of classic unsigned array multiplier.

Combinational Block	No. of Applied Blocks	No. of Computational Steps
PPU_1	$n - 1$	22
PPU_2	$n^2 - 4n + 5$	27
PPU_3	$n - 2$	32
Half Adder [28]	$n - 1$	1
Full Adder [23]	$n - 3$	22
AND Gate	1	5

Overlapping repeated serial steps in the arithmetic cell's implementation consisting of several primary blocks reduces the computational steps and hardware complexity (reducing the number of memristors) in IMPLY-based serial designs [28]. In PPU_1, the outputs of two AND gates are the inputs of the half adder cell. In the classic structure, the number of computational steps of PPU_1 equals the sum of the computational steps of two logical AND gates (10 steps) and a half adder cell (12 steps). Thus, the total computational steps of classic PPU_1 are 22 steps.